

# Gravar e depurar na Blue Pill (STM32F103C8T6)

Guia dos passos definidos para levar o firmware do build até a execução na placa, via ST-Link. Vale para o projeto bare-metal deste repositório (CMake + Ninja + toolchain `arm-none-eabi` do bundle STM32Cube).

⚠ **Estado atual do firmware:** `Src/main.c` é apenas um `for(;;)` vazio — o "blink" ainda não foi implementado. Gravar/rodar valida toolchain + ST-Link, mas **o LED (PC13) não vai piscar** até o GPIO ser configurado.

## Ferramentas confirmadas neste ambiente

| Ferramenta              | Caminho                                                                                                      |
|-------------------------|--------------------------------------------------------------------------------------------------------------|
| STM32CubeProgrammer CLI | <code>C:\ST\STM32CubeCLT_1.21.0\STM32CubeProgrammer\bin\STM32_Programmer_CLI.exe</code>                      |
| OpenOCD (xpack 0.12.0)  | <code>F:\tools\xpack-openocd-0.12.0-7\bin\openocd.exe</code> (scripts em <code>...\openocd\scripts\</code> ) |
| arm-none-eabi-gdb       | <code>C:\ST\STM32CubeCLT_1.21.0\GNU-tools-for-STM32\bin\arm-none-eabi-gdb.exe</code>                         |
| SVD do STM32F103        | <code>C:\ST\STM32CubeCLT_1.21.0\STMicroelectronics\CMSIS_SVD\STM32F103.svd</code>                            |
| Bundle STM32Cube        | <code>C:\Users\muril\AppData\Local\stm32cube\bundles</code> ( <code>CUBE_BUNDLE_PATH</code> )                |

Artefatos de build (após compilar): `build/Debug/blink_bluepill.{elf,hex,bin}`. Endereço de flash: `0x08000000`.

## Passo 1 — Build

CMake + Ninja, presets `Debug` (`-O0 -g3`) e `Release` (`-Os`):

```
cmake --preset Debug
cmake --build --preset Debug
```

Ou pela extensão **CMake Tools** no VS Code: selecione o preset `Debug` na barra de status inferior e clique em **Build** (`F7`). Saída em `build/Debug/`.

## Passo 2 — Gravar o firmware

Conexão (SWD) — comum a todos os métodos

| ST-Link V2 | Blue Pill   |
|------------|-------------|
| SWDIO      | SWDIO (DIO) |
| SWCLK      | SWCLK (CLK) |
| GND        | GND         |
| 3.3V       | 3.3V        |

Método A — `STM32_Programmer_CLI` (mais à prova de falhas)

```
& "C:\ST\STM32CubeCLT_1.21.0\STM32CubeProgrammer\bin\STM32_Programmer_CLI.exe" `
  -c port=SWD -w "build\Debug\blink_bluepill.elf" -rst
```

Gravando o `.bin` (endereço obrigatório): adicione `0x08000000` após o caminho. Flags: `-c port=SWD` conecta · `-w` grava · `-v` verifica · `-rst` reseta e roda.

## Método B — OpenOCD

```
& "F:\tools\xpack-openocd-0.12.0-7\bin\openocd.exe" `
  -f interface/stlink.cfg -f target/stm32f1x.cfg `
  -c "program build/Debug/blink_bluepill.elf verify reset exit"
```

## Método C — Extensão STM32Cube no VS Code (Run and Debug)

Este projeto usa o **pacote STM32CubeIDE para VS Code** (não a extensão antiga com barra lateral "Build/Run/Debug"). O fluxo é:

- **Build** → extensão CMake Tools (passo 1).
- **Flash + Debug** → painel **Run and Debug** (`Ctrl+Shift+D`) com um `launch.json` do tipo `stlinkgdbtarget`.

Se não houver `launch.json`: abra Run and Debug → "create a launch.json file" → escolha **"STM32Cube: STM32 Launch ST-Link GDB Server"**. A configuração usada neste projeto (`.vscode/launch.json`):

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "stlinkgdbtarget",
      "request": "launch",
      "name": "STM32Cube: Launch ST-Link GDB Server",
      "origin": "snippet",
      "cwd": "${workspaceFolder}",
      "preBuild": "${command:st-stm32-ide-debug-launch.build}",
      "runEntry": "main",
      "svdPath": "C:/ST/STM32CubeCLT_1.21.0/STMicroelectronics_CMSIS_SVD/STM32F103.svd",
      "serverVerify": true,
      "imagesAndSymbols": [
        {
          "imageFileName": "${workspaceFolder}/build/Debug/blink_bluepill.elf",
          "symbolFileName": "${workspaceFolder}/build/Debug/blink_bluepill.elf"
        }
      ]
    }
  ]
}
```

Pontos-chave dessa config:

- `symbolFileName` → carrega símbolos (breakpoints por linha / debug de fonte).
- `imageFileName` com caminho explícito → mais confiável que o comando de contexto em projeto CMake puro.  
Alternativa preset-aware: `${command:cmake.launchTargetPath}`.

- `svdPath` → habilita o viewer de periféricos (extensão `stm32cube-ide-registers`).
- `serverVerify: true` → verifica a gravação após o download.

## Passo 3 — Rodar / depurar

Com o ST-Link conectado, selecione "**STM32Cube: Launch ST-Link GDB Server**" no painel Run and Debug e pressione **F5**. Esperado:

1. `preBuild` dispara o build,
2. ST-Link GDB Server grava o `.elf` em `0x08000000`,
3. execução para em `main` (por `"runEntry": "main"`).

Controles: Continue (**F5**), Step Over (**F10**), Step Into (**F11**). A Debug Console mostra logs de download/verify. Para os comandos GDB úteis no console (`>show version`, `>info registers`, `>bt`, etc.), veja [debug-console-gdb.md](#).

### Alternativa sem ST-Link: bootloader serial

Adaptador USB-Serial 3.3V no UART1 (PA9/PA10): jumper `BOOT0=1` + RESET → grava com `STM32_Programmer_CLI -c port=COMx -w firmware.bin 0x08000000` → volte `BOOT0=0` + RESET.

## Troubleshooting

Console de Debug mostra dica do GDB e `help/>help` não respondem

Ao iniciar o debug, a Debug Console exibe:

```
In the Debug Console view you can interact directly with GDB.
To display the value of an expression, type that expression ...
Arbitrary commands can be sent to GDB by prefixing the input with a '>',
for example type '>show version' or '>help'.
```

Isso é **informativo, não é erro** — é o adaptador avisando como usar o console.

- Texto **sem** prefixo é avaliado como **expressão** (ex.: `2 + 3`, ou o nome de uma variável em escopo). Por isso `help` (sem `>`) tende a falhar/retornar nada: não é uma expressão válida e não há variável chamada `help`.
- Comandos do GDB precisam do prefixo `>`. Ex.: `>help`, `>show version`, `>info registers`.

**Causa comum de `>help` parecer "não responder"**: os comandos GDB só funcionam quando o alvo está **parado** (**halted**) e a sessão de debug está ativa/conectada. Se a CPU está rodando (após Continue) ou a sessão ainda não conectou, o GDB fica ocupado e não processa o comando. Solução:

1. Confirme que a sessão de debug está ativa e **pausada** (deve ter parado em `main`, ou clique em **Pause/Halt** na barra de debug).
2. Então digite `>help` (com o `>`) na Debug Console.
3. Para ver registradores do core: `>info registers`. Para periféricos (RCC/GPIO), use o **viewer de periféricos** (habilitado pelo `svdPath`), não o console.

"Cube programmer not found" ao iniciar o GDB server

O `serverCubeProgPath` está vazio (auto-localização). Se falhar, defina no `launch.json`: `"serverCubeProgPath": "C:/ST/STM32CubeCLT_1.21.0/STM32CubeProgrammer/bin"`.

`imageFileName` não resolve / nada é gravado

Se tiver voltado ao comando `${command:...get-projects-binary-from-context1}` e ele não resolver em projeto CMake, troque pelo caminho explícito do `.elf` (como acima) ou por `${command:cmake.launchTargetPath}`.

## Depurando o preset Release

O caminho no `imageFileName` está fixo em `build/Debug/...`. Para depurar `Release`, ajuste para `build/Release/...` ou use `${command:cmake.launchTargetPath}` (segue o preset ativo do CMake Tools).

## Sem breakpoints por linha / "no source"

Garanta que `symbolFileName` aponta para o `.elf` (não o `.bin`) e que o build foi feito no preset `Debug` (`-g3`). O `.bin/.hex` não contém símbolos.

## Placa não detectada / falha de conexão SWD

Verifique a fiação SWD (incl. GND e 3.3V), atualize o firmware do ST-Link (comando `Upgrade ST-Link firmware` da extensão) e use `Refresh device list`. Em casos de SWD travado, tente `connect under reset` (default da config) ou segure RESET ao conectar.